Modified K-means Algorithm for Vector Quantizer Design

Daeryong Lee, Seongjoon Baek, and Koengmo Sung, *Member, IEEE*

Abstract— **The K-means algorithm is widely used in vector quantizer (VQ) design and clustering analysis. In VQ context, this algorithm iteratively updates an initial codebook and converges to a locally optimal codebook in certain conditions. It iteratively satisfies each of the two necessary conditions for an optimal quantizer; the nearest neighbor condition for the partition and centroid condition for the codevectors. In this letter, we propose a new algorithm for both vector quantizer design and clustering analysis as an alternative to the conventional K-means algorithm. The algorithm is almost the same as the K-means algorithm except for a modification at codebook updating step. It does not satisfy the centroid condition iteratively, but asymptotically satisfies it as the number of iteration increases. Experimental results show that the algorithm converges to a better locally optimal codebook with an accelerated convergence speed.**

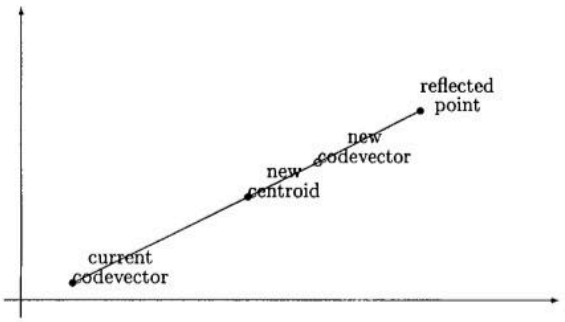


Fig. 1. New codebook updating method.

I. Introduction

VECTOR quantization (VQ) is a mapping of input data to one of a finite number of prestored codevectors (or codewords) in a codebook [1], [2]. VQ is widely used in image and speech compression. The goal in VQ design is to obtain an optimal codebook that minimizes the average distortion introduced by the approximation of each input data to one of the codevectors. Such an optimal quantizer must satisfy two necessary conditions, the nearest neighbor condition for the partition and centroid condition for the codevectors [2], [3].

To date, the most widely used algorithm to design a VQ codebook is the K-means algorithm. It is also known as the generalized Lloyd algorithm (GLA), or sometimes called the Linde-Buzo-Gray (LBG) algorithm [1]–[4]. It updates an initial codebook and satisfies the two necessary conditions iteratively. The algorithm consists of the following sequence of steps [4]:

Step 1) Initial condition: Begin with any desired initial configuration. Go to step 2) if beginning with a set of codevectors (initial codebook); go to step 3) if beginning with a partition of the data.

Step 2) Partitioning: Allocate each data to the cluster with the nearest codevector.

Step 3) Codebook updating: Compute new codevectors as the centroids of the clusters of data.

Step 4) Convergence check: Alternate step 2) and step 3) until the process converges.

It is known that K-means algorithm converges to a locally optimal codebook in certain conditions [5] but it converges to a different codebook when a different initial condition is applied. It has also been observed that both convergence speed and performance of the converged codebook depend on the initial codebook. Thus, many algorithms were proposed to obtain a good initial codebook, including the well known splitting, pruning, pairwise nearest neighbor design (PNN), random initialization and the newly proposed maximum distance initialization [6]. PNN and maximum distance algorithm were also known in clustering analysis [4][7].

Zeger *et al.* [3] proposed other types of VQ algorithm using stochastic relaxation such as simulated annealing to get a better codebook than K-means algorithm. They introduced the concepts of encoder pertubation and decoder pertubation. Encoder pertubation is implemented by corrupting the input training data by an additive noise component prior to each nearest neighbor repartitioning at each iteration. Decoder pertubation is implemented by applying noise to the reconstructed codevectors. By reducing the amount of pertubations at each iteration, these methods showed performance improvement over K-means algorithm for image and speech data. These algorithms nevertheless require significantly more computation time than K-means algorithm.

In this letter, we propose a new VQ design algorithm called *modified K-means algorithm* as an alternative to the well-known K-means algorithm. It results in a better locally optimal codebook than K-means algorithm with the same initial codebook. It also iteratively updates the initial codebook like the conventional K-means algorithm but with a somewhat different way to be explained. The computation time for each iteration is almost the same as the K-means algorithm, but the modified K-means algorithm generally demands fewer iterations for convergence. It can also be thought of as a kind of decoder pertubation of Zeger *et al.* with almost no additional computational burden.

Manuscript received April 3, 1996. The associate editor coordinating the review of this manuscript and approving it for publication was Prof. R. Ansari.

The authors are with the Department of Electronics Engineering, Seoul National University, Seoul 151-742, Korea (e-mail: ldr@ctlab2.snu.ac.kr).

Publisher Item Identifier S 1070-9908(97)01275-3.

1070–9908/97$10.00 © 1997 IEEE

Authorized licensed use limited to: UNICAP - Universidade Católica de Pernambuco. Downloaded on October 19,2026 at 12:18:11 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PERFORMANCE COMPARISON FOR VARIOUS VALUES OF *SCALE* WITH MAXIMUM DISTANCE INITIALIZATION FOR LENA IMAGE DATA WITH 512 CODEBOOK

scale	0.8	0.9	1.0	1.1	1.2	1.3	1.4	1.5	1.6	1.7	1.8	1.9	2.0
MSE	33.266	33.497	33.341	33.199	33.086	32.753	32.726	32.540	32.348	32.178	32.185	32.290	37.241
iteration	28	22	20	23	24	20	16	18	19	20	17	25	39

TABLE II
PERFORMANCE COMPARISON WITH MAXIMUM DISTANCE INITIALIZATION
(itr: num of iteration, CBsize: codebook size)

CBsize	256						512						1024					
scale	1.0(K-means)		2.0(Jancey)		1.8(new)		1.0		2.0		1.8		1.0		2.0		1.8	
	MSE	itr	*MSE*	itr	*MSE*	itr	*MSE*	itr	*MSE*	itr	*MSE*	itr	*MSE*	itr	*MSE*	itr	*MSE*	itr
lena	44.328	28	47.459	45	42.769	20	33.341	20	37.241	39	32.185	17	23.153	16	25.952	29	22.510	19
baboon	249.506	21	249.905	28	246.976	20	211.844	16	216.029	22	207.401	17	172.300	15	182.720	21	168.463	16
pepper	47.962	21	52.941	49	46.876	18	36.453	22	39.923	33	32.670	17	25.629	14	28.255	30	24.888	16

II. MODIFIED K-MEANS ALGORITHM

Our proposed algorithm is almost the same as the conventional K-means algorithm except for a modification at new codebook updating step. The first set of codevectors is either given or computed as the centroids of clusters in the initial partition; at all succeeding stages each new codevector is selected not by the new centroid for the cluster, but by one of the points in the line connecting the old codevector with the reflected point of the old codevector through the new centroid for the cluster. As shown in Fig. 1, the K-means algorithm chooses point 2, the new centroid for the cluster, as the new codevector. But we think intuitively that any point between points 1 and 3 can be chosen as the new codevector for the cluster. If one of the points between points 1 and 2 is chosen as the new codevector, the convergence speed will surely be slow, but if one of the points between points 2 and 3 is chosen as the new codevector, the algorithm may converge fast. The line from point 1 to point 2 can be viewed as an approximation to the local gradient, the direction in which the codevector should move for the greatest improvement in the partition [4]. We hope that further movement in that direction will ultimately lead to a convergence with a smaller value of average distortion with an accelerated speed. So we propose to choose one of the points between points 2 and 3 as the new codevector for the cluster. We suggest the new codevector be chosen as *new codevector* $=$ *old codevector* $+$ *scale* $\times$ (*new centroid* $-$ *old codevector*). The proposed algorithm demands almost the same computation time as the K-means algorithm for each iteration. In clustering analysis, Jancey used the point 3 as the new seed point (new codevector in VQ context), which corresponds in our method when the value of scale is 2.0 [4]. But we intuitively think that the region of convergence is inside the circle, which is made by the line connecting point 1 with point 3 as a diameter. Because the new codevector of Jancey's method is on the boundary of convergence, it does not guarantee convergence. Even if it converges it may take too many iterations. The possibility of nonconvergence arises because there is no apparent barrier to repeating some partitions and cycling through them again and again. If groupings about points 1 and 3 in Fig. 1 give the same membership, the codevector can oscillate between the two points instead of going to point 2, which will give better performance [4]. But if we choose one of the points between points 2 and 3 as the new codevector, there will be no problems such as Jancey's and will result in a better codebook than K-means algorithm or Jancey's with an accelerated convergence speed. The centroid condition is asymptotically satisfied because the length of the line connecting point 1 with point 2 gradually decreases as the number of iteration increases. In our experiment when the value of scale is about 1.8, the algorithm generally achieves good performance results.

III. EXPERIMENTAL RESULTS

Two kinds of experiment were performed. One was to show the effect of the value of scale of the proposed algorithm. The other was to compare the performance of the proposed algorithm with those of the two previous ones, the conventional K-means algorithm and Jancey's method.

In these experiments, we used typical method of coding still images. We divided each image into 16 384 blocks of pixels. Each block is 4×4 pixels insize. The images are 512×512 monochrome USC images with 256 gray levels, Lena, baboon and pepper. We experimented with different codebook sizes of 256, 512, and 1024. The quality of the coded images is evaluated by mean-square error (MSE) which is defined as

$$MSE = \left(\frac{1}{512}\right)^2 \sum_{i=1}^{512}\sum_{j=1}^{512}(x_{ij} - y_{ij})^2 \qquad (1)$$

where x_{ij} is the pixel value of original image at coordinate (i, j) and y_{ij} is the pixel value of coded image at coordinate (i, j).

The algorithms are stopped when the ratio of the MSE difference and the MSE of the current is within 0.0005 or 0.05%. In Table I, the values of MSE and iterations for the Lena image with 512 codebook for various values of *scale* are shown. As expected, as the value of *scale* is increasing, the algorithm converges faster and shows better performance. But as the value of *scale* approaches 2.0, the algorithm needs abruptly more iterations and shows poor performance.

Authorized licensed use limited to: UNICAP - Universidade Católica de Pernambuco. Downloaded on October 19,2026 at 12:18:11 UTC from IEEE Xplore. Restrictions apply.

TABLE III
PERFORMANCE COMPARISON WITH SPLITTING INITIALIZATION
(itr: number of iteration, CBsize: codebook size)

CBsize	256						512						1024					
scale	1.0(K-means)		2.0(Jancey)		1.8(new)		1.0		2.0		1.8		1.0		2.0		1.8	
	MSE	itr	MSE	itr	MSE	itr	MSE	itr	MSE	itr	MSE	itr	MSE	itr	MSE	itr	MSE	itr
lena	42.663	20	43.341	19	41.808	18	32.983	16	34.757	19	31.903	15	24.575	14	28.401	18	23.483	15
baboon	250.716	18	247.904	15	247.871	13	211.894	14	210.678	17	207.899	15	176.312	14	179.576	16	169.509	14
pepper	46.004	19	46.768	18	45.146	16	36.429	16	38.218	15	34.411	16	27.223	10	31.767	15	25.473	15

Performance is relatively better when the value of *scale* is between 1.7 and 1.9. Considering both the convergence speed and performance, we propose the value of *scale* be about 0.8. The performance results when the value of *scale* is 1.0 (K-means algorithm), 2.0 (Jancey's method) and 1.8 (proposed) are shown in Tables II and III for splitting and maximum distance initialization, respectively. We can see that the proposed method gives better performance than the K-means algorithm and Jancey's method in all cases.

IV. CONCLUSION

A new vector quantizer design algorithm has been proposed in this letter. It is almost the same as the conventional K-means algorithm except for a modification at the codebook updating step. It generally demands fewer iterations and achieves a better locally optimal codebook than the conventional K-means algorithm. It thus provides an efficient alternative to the conventional K-means algorithm for both vector quantizer design and clustering analysis.

REFERENCES

[1] Y. Linde, A. Buzo, and R. M. Gray, "An algorithm for vector quantizer design.," *IEEE Trans. Commun.*, vol. COM-28, pp. 84–95, Jan. 1980.
[2] A. Gersho and R. M. Gray, *Vector Quantization and Signal Compression*. Boston, MA: Kluwer, 1992.
[3] K. Zeger, J. Vaisey, and A. Gersho, "Globally optimal vector quantizer design by stochastic relaxation," *IEEE Trans. Signal Processing*, vol. 40, pp. 310–322, Feb. 1992.
[4] M. R. Anderberg, *Cluster Analysis for Applications*. New York: Academic, 1973.
[5] S. Z. Selim and M. A. Ismail, "K-means-type algorithms: A generalized convergence theorem and characterization of local optimality," *IEEE Trans. Pattern Anal. Machine Intell.*, vol. PAMI-6, pp. 81–87, Jan. 1984.
[6] I. Katsavounidis, C.-C. J. Kuo, and Z. Zhang, "A new initialization technique for generalized Lloyd iteration," *IEEE Signal Processing Lett.*, vol. 1, pp. 144–146, Oct. 1994.
[7] S. T. Bow, *Pattern Recognition and Image Preprocessing*. New York: Marcel Dekker, 1992.

Correspondence

Comments on "Modified K-Means Algorithm for Vector Quantizer Design"

Kuldip K. Paliwal and V. Ramasubramanian

***Abstract*—Recently a modified K-means algorithm for vector quantization design has been proposed where the codevector updating step is as follows: new codevector = current codevector + scale factor (new centroid − current codevector). This algorithm uses a fixed value for the scale factor. In this paper, we propose the use of a variable scale factor which is a function of the iteration number. For the vector quantization of image data, we show that it offers faster convergence than the modified K-means algorithm with a fixed scale factor, without affecting the optimality of the codebook.**

***Index Terms*—Faster convergence, K-means algorithm, vector quantization.**

I. Introduction

Vector quantization is a powerful data compression technique used in a number of signal processing applications including speech coding, image coding, and speech recognition [1]–[6]. Design of a vector quantizer is accomplished by generating a codebook from the training data using a distortion measure appropriate for the given application. The K-means clustering algorithm (or the generalized Lloyd algorithm) is usually used for this purpose [7], [8]. This algorithm is iterative in nature and requires a large amount of computation time for convergence. The computation time mainly depends on the amount of training data, codebook size, vector dimension, and distortion measure.

Recently, a modified K-means algorithm for vector quantization design was proposed by Lee *et al.* [9] for faster convergence. In this algorithm, the codevector updating step is as follows: new codevector = current codevector + scale factor (new centroid − current codevector). This algorithm uses a fixed value for the scale factor. In this paper, we propose the use of a variable scale factor which is a function of the iteration number. For vector quantization of image data, we show that it offers faster convergence than the modified K-means algorithm with a fixed scale factor, without affecting the optimality of the codebook.

II. Modified K-Means Algorithm

Recently, Lee *et al.* [9] proposed a modified K-means algorithm which provides faster convergence rate than the conventional K-means algorithm [7]. This algorithm achieves acceleration in the convergence by using a "scaled" updating scheme where a codevector is updated along the direction of the local gradient by a step-size larger than that used by the centroid update of the conventional K-means algorithm. First, we briefly describe this "scaled" update scheme with respect to the conventional K-means algorithm.

Manuscript received December 3, 1997; revised June 5, 2000. The associate editor coordinating the review of this manuscript and approving it for publication was Dr. Antonio Ortega.

K. K. Paliwal is with the School of Microelectronic Engineering, Griffith University, Brisbane, QLD 4111, Australia (e-mail: K.Paliwal@me.gu.edu.au).

V. Ramasubramanian is with the Tata Institute of Fundamental Research, Bombay 400005, India.

Publisher Item Identifier S 1057-7149(00)09392-1.

The conventional K-means algorithm [7] for the design of a codebook $\mathbf{C} = \{\mathbf{c}_i, i = 1, \cdots, N\}$ of size N from the training data $T = \{\mathbf{x}_m, m = 1, \cdots, M\}$ is as follows.

1) *Initialization:* Iteration number $n = 0$; codebook at iteration n, $\mathbf{C}^n = \{\mathbf{c}_i^n, i = 1, \cdots, N\}$; convergence threshold ϵ.
2) *Partitioning:* Find the nearest-neighbor partition $V(\mathbf{c}_j^n) = \{\mathbf{x}_m \in T: Q(\mathbf{x}_m) = \mathbf{c}_j^n\}$, $j = 1, \cdots, N$. Here, Q denotes vector quantization operation and is defined as follows: $Q(\mathbf{x}) = \mathbf{c}_j^n$ if $d(\mathbf{x}, \mathbf{c}_j^n) \leq d(\mathbf{x}, \mathbf{c}_i^n)$, $i = 1, \cdots, N$ under some distance measure $d(\mathbf{x}, \mathbf{y})$, $\mathbf{x}, \mathbf{y} \in R^K$.
3) *Codebook Update:* Update codevectors $\mathbf{C}^n = \{\mathbf{c}_j^n, j = 1, \cdots, N\}$ to $\mathbf{C}^{n+1} = \{\mathbf{c}_j^{n+1}, j = 1, \cdots, N\}$ as
$$\mathbf{c}_j^{n+1} = \mathcal{C}(V(\mathbf{c}_j^n)) \tag{1}$$
where $\mathcal{C}(V(\mathbf{c}_j^n))$ is the centroid of the partition $V(\mathbf{c}_j^n)$ under the given distance measure $d(\mathbf{x}, \mathbf{y})$.
4) *Convergence Check:* Stop if $|d_n - d_{n-1}|/d_n \leq \epsilon$, where $d_n = (1/M)\sum_{m=1}^{M} d(\mathbf{x}_m, Q(\mathbf{x}_m))$. Otherwise, replace n by $n + 1$ and go to Step 2.

The modified K-means algorithm proposed by Lee *et al.* [9] updates the current codevector $\mathbf{c}_j^n$ (at iteration n) to the new codevector $\mathbf{c}_j^{n+1}$ to be used at iteration $n + 1$ as
$$\mathbf{c}_j^{n+1} = \mathbf{c}_j^n + s(\mathcal{C}(V(\mathbf{c}_j^n)) - \mathbf{c}_j^n). \tag{2}$$
Under a squared-error distance measure, Lee *et al.* [9] have found experimentally that the modified K-means algorithm converges slower in comparison to the conventional K-means algorithm when $s < 1$. When $1 < s < 2$, it converges faster and results in better performance in terms of mean-squared error. When $s > 2$, the algorithm either does not converge, or converges very slowly with poor performance. The modified K-means algorithm gives the best results when the scale factor is set to a fixed value of $s = 1.8$. It may be noted that for $s = 1$, the modified K-means algorithm becomes the same as the conventional K-means algorithm with centroid-update as given by (1).

III. Variable Scale Update

The codevector updates obtained by the conventional centroid-update [see (1)] have progressively decreasing step-sizes [given by $d(\mathbf{c}_j^n, \mathbf{c}_j^{n+1})$] as the codevectors encode the training data with decreasing mean squared error at each iteration until convergence when the codevectors do not have any appreciable update. Therefore, while the use of a scaled-update as in (2) can accelerate the convergence, use of a "fixed" scaling for the entire range of iterations results in the use of step sizes larger than the corresponding centroid-update at iterations closer to convergence and causes undesirably high perturbations of the codevectors which are otherwise converging to some optimal configuration. This in turn has the effect of increasing the number of iterations required to converge as well as perturbing the codebook convergence to a poorer local optimum. This can also be seen from the convergence characteristics observed in [9] and [10] for fixed scale values larger than 2, where either the convergence is very slow or nonexistent due to the step-size being considerably larger than the conventional centroid-update.

1057–7149/00$10.00 © 2000 IEEE

Authorized licensed use limited to: UNICAP - Universidade Católica de Pernambuco. Downloaded on October 19,2026 at 12:24:56 UTC from IEEE Xplore. Restrictions apply.

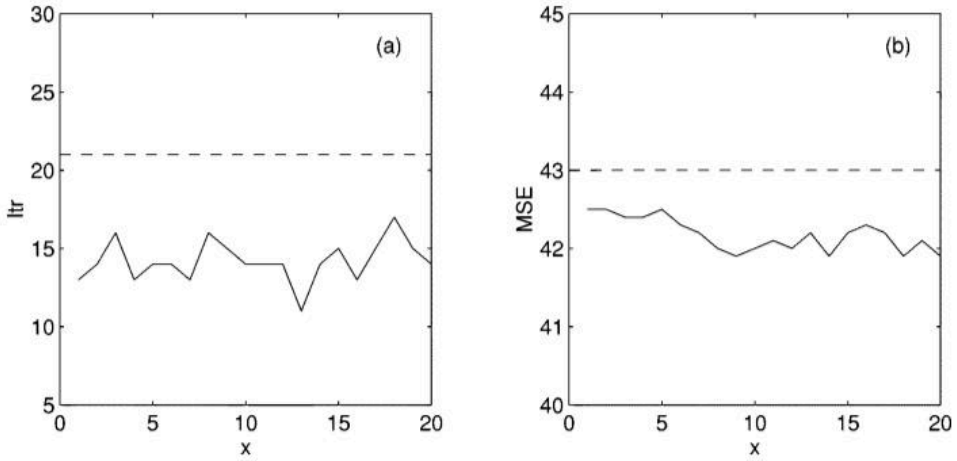


Fig. 1. (a) Number of iterations (Itr) required to reach convergence and (b) MSE at convergence for the NEW algorithm as a function of x (shown in solid line) on the Lena image data with dimension $K = 16$ and codebook size $N = 256$. Dashed horizontal lines show Itr and MSE for the CONV algorithm.

TABLE I
PERFORMANCE COMPARISON OF THE ALGORITHMS (CONV, MODI, AND NEW) FOR THE IMAGE DATA SET I

Initialization by minmax method							
Image	Alg	Codebook Size (N)					
		256		512		1024	
		Itr	MSE	Itr	MSE	Itr	MSE
Lena	CONV	25	44.4	23	33.1	15	24.6
	MODI	18	42.8	17	32.1	16	23.1
	NEW	18	42.9	15	32.4	12	23.2
Pepper	CONV	23	49.4	17	36.3	12	27.3
	MODI	18	47.7	16	35.1	16	25.8
	NEW	14	48.1	14	35.1	11	25.8
Baboon	CONV	21	250.5	16	211.4	13	173.6
	MODI	18	247.1	16	207.7	15	168.9
	NEW	19	248.2	12	209.0	11	168.8
Initialization by splitting method							
Image	Alg	Codebook Size (N)					
		256		512		1024	
		Itr	MSE	Itr	MSE	Itr	MSE
Lena	CONV	21	43.0	15	33.2	15	24.7
	MODI	19	42.2	14	32.2	15	23.8
	NEW	15	41.9	12	32.3	12	23.7
Pepper	CONV	16	46.9	14	36.3	14	27.4
	MODI	14	46.1	15	35.4	16	26.5
	NEW	12	46.0	10	35.9	11	26.3
Baboon	CONV	17	250.6	14	213.4	12	176.4
	MODI	15	248.5	14	208.9	13	171.0
	NEW	12	248.1	11	210.0	11	171.6

TABLE II
PERFORMANCE COMPARISON OF THE ALGORITHMS (CONV, MODI, AND NEW) FOR THE IMAGE DATA SET II

Initialization by minmax method									
Alg	Codebook Size (N)								
	256		512		1024		2048		4096	
	Itr	MSE	Itr	MSE	Itr	MSE	Itr	MSE	Itr	MSE
CONV	31	149.9	29	126.1	23	104.6	20	85.8	16	68.1
MODI	19	148.4	24	123.3	22	101.8	18	83.2	15	65.8
NEW	20	148.5	20	123.9	16	103.1	20	83.4	13	65.9
Initialization by splitting method									
Alg	Codebook Size (N)								
	256		512		1024		2048		4096	
	Itr	MSE	Itr	MSE	Itr	MSE	Itr	MSE	Itr	MSE
CONV	20	148.1	25	123.7	18	103.1	15	84.9	14	68.5
MODI	21	146.9	19	122.4	18	101.5	15	83.2	14	66.6
NEW	17	148.1	19	123.2	14	102.2	12	83.6	12	66.9

Therefore, in order to obtain codevector updates for iteration $n + 1$, which are reliable look-ahead estimates of conventional codevector updates corresponding to iterations $n + 2$, $n + 3$, $\cdots$, the scale factor in (2) should decrease with increase in iteration. Here, we propose the use of a scale factor s in the codevector update in (2) which varies as a function of the iteration n and is inversely proportional to n. This scale factor should satisfy the following two conditions. 1) It should be greater than 1 to ensure a faster convergence than the conventional K-means algorithm [9] and 2) it should be less than 2 to avoid a very slow or nonexistent convergence [9], [10]. A functional form (s as a function of n) that meets these requirements is as follows:

$$s = 1 + \frac{x}{(x + n)} \tag{3}$$

where $x > 0$. In this equation, $s = 2$ when $n = 0$, and $s = 1$ when $n = \infty$; thus, it satisfies the aforementioned conditions. Note that this type of functional dependence of the scale factor on the iteration number has been used earlier in a number of algorithms reported in the literature; e.g., algorithms used for stochastic approximation [11], [12], self-organizing feature map generation [13], simulated annealing [14], etc. Also, note that this is not the only functional form that satisfies the above-mentioned requirements. Other functional forms are also possible and may be investigated in future.

IV. SIMULATION RESULTS

We compare the convergence characteristics of the three K-means clustering algorithms, namely

1) CONV, which uses the conventional update given by (1);
2) MODI, which uses the modified update given by (2) with a fixed scale value of $s = 1.8$;
3) NEW, the algorithm proposed here, which uses a variable scale factor $s = 1 + x/(x + n)$ (where n is the iteration number).

The three K-means algorithms are also compared with two initializations as in [9]: 1) maximum distance initialization using the minmax method [15] and 2) splitting initialization [7].

The performance of the K-means algorithms are studied here under squared-error distance measure for the vector quantization for image

Authorized licensed use limited to: UNICAP - Universidade Católica de Pernambuco. Downloaded on October 19,2026 at 12:24:56 UTC from IEEE Xplore. Restrictions apply.

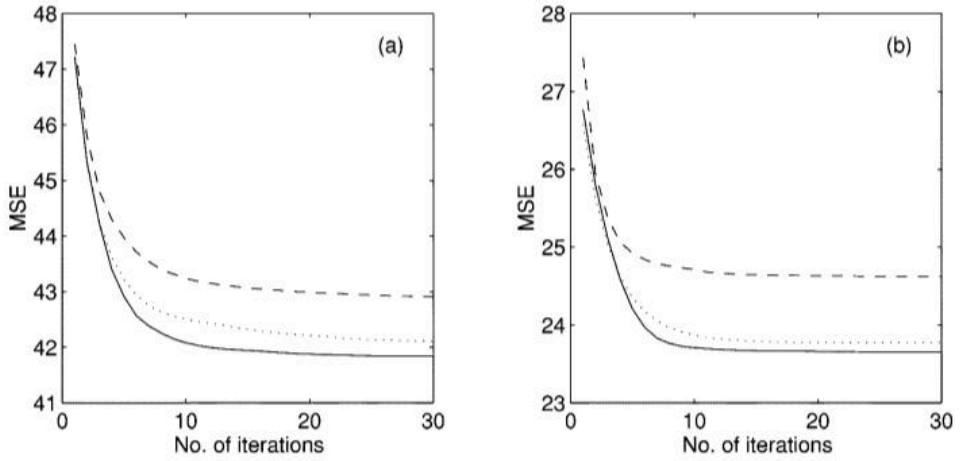


Fig. 2. MSE as a function of number of iterations for the CONV algorithm (dashed line), the MODI algorithm (dotted line) and the NEW algorithm (solid line) on the Lena image data with dimension $K = 16$. (a) Codebook size $N = 256$ and (b) Codebook size $N = 1024$.

data. For this we have used two sets of image data: 1) Set-I: Lena, Baboon and Pepper used separately as in [9] and 2) Set-II: A single data set consisting of the images (6386, baboon, bank, c43m, face, hat, vegas). Each of these monochrome images has 512×512 pixels with 256 gray levels.

We divide the images into a number of disjoint blocks, each of size 4×4 pixels and each resulting in a 16-dimensional vector. Different clustering algorithms are used to generate codebooks of sizes 256, 512, 1024, 2048, and 4096. These codebooks are then used to vector quantize the images. The quality of the coded image is evaluated in terms of mean-squared error (MSE) which is defined as

$$\text{MSE} = \frac{1}{IJ} \sum_{i=1}^{I} \sum_{j=1}^{J} (x_{ij} - y_{ij})^2, \tag{4}$$

where $I \times J$ is the size of the image and x_{ij} and y_{ij} are the pixel values of the original and coded images, respectively, at the coordinate (i, j). As done in [9], we also stop the algorithm and consider the convergence to be reached when the ratio of the MSE difference between the current and the preceding iterations and the MSE of the current iteration is less than 0.0005 or 0.05%.

Since the computation time for each iteration is almost the same for all the three algorithms, we use the number of iterations (Itr) needed to reach the convergence as a measure of the computational cost of an algorithm. The value of MSE at convergence is used as a measure of the optimality of the designed codebook. The same measures have been used by Lee *et al.* [9] to characterize the performance of the K-means algorithms.

In order to see the effect of variable x used in the scale factor equation $s = 1 + x/(x + n)$, we have studied the NEW algorithm with various values of x. Fig. 1 shows number of iterations (Itr) needed to reach convergence and the value of MSE at convergence as a function of x. As a reference, we also show in this figure by dashed horizontal lines the values of Itr and MSE for the CONV algorithm. It can be seen that the NEW algorithm performs better than the CONV algorithm for all the values of x. This is consistent with the results obtained by Lee *et al.* [9], as the scale factor in the NEW algorithm always satisfies the condition $1 < s < 2$ for all x. Also, note that the NEW algorithm works equally well for a wide range of x values. For the results reported hereafter in this paper, we use $x = 9$.

In Table I, we show the number of iterations (Itr) required for convergence and MSE for images from Set-I for codebook sizes $N = 256, 512, 1024$ and dimension $K = 16$ (using blocks of 4×4 pixels) using training data of 16 384 vectors. In Table II, we have used data from set-II for codebook sizes $N = 256, 512, 1024, 2048$ and 4096 and dimension $K = 16$ with 114 688 vectors. In order to illustrate the convergence behavior of the three algorithms, we show in Fig. 2 the MSE as a function of number of iterations for the Lena image with dimension $K = 16$ and codebook sizes $N = 256$ and 1024.

From these two tables and the figure, we can make the following observations: 1) the modified K-means algorithm (MODI), which uses a fixed scale value of 1.8, provides most of the times (but not always) faster convergence than the conventional K-means algorithm (CONV) and 2) the K-means algorithm proposed here, NEW, which updates codevectors using a variable scale function, always provides a faster convergence than the CONV algorithm. Also note that the NEW algorithm does not sacrifice the optimality of the codebook with respect to the CONV algorithm in the sense that it does not increase the MSE. In fact, it reduces the MSE compared to the conventional K-means algorithm, though this reduction may not be significant. It is important to note here that the modification proposed in this paper is very simple and easy to implement.

We have reported here results for dimension $K = 16$. However, we have also carried out experiments for dimensions $K = 4$ and 64. These experiments show results similar to those described above for the three K-means algorithms.

V. Conclusions

In this paper, we have proposed the use of a modified update step based on a variable scale size which is a function of the iteration and shown that it offers faster convergence than the modified K-means algorithm with a fixed scale update without effecting the optimality of the codebook. Though the scale factor in the proposed algorithm varies as a function of iteration number, it is same for all the components of a vector in a given iteration. In principle, it is possible to assign different scale values for different components of a vector according to some criterion which might improve the convergence behavior further. However, this has to be investigated in the future.

References

[1] H. Abut, Ed., *Vector Quantization*. Piscataway, NJ: IEEE Press, May 1990.
[2] A. Gersho and R. M. Gray, *Vector Quantization and Signal Compression*. Norwell, MA: Kluwer, 1992.
[3] W. B. Kleijn and K. K. Paliwal, Eds., *Speech Coding and Synthesis*. Amsterdam, The Netherlands: Elsevier, 1995.

Authorized licensed use limited to: UNICAP - Universidade Católica de Pernambuco. Downloaded on October 19,2026 at 12:24:56 UTC from IEEE Xplore. Restrictions apply.

[4] K. K. Paliwal and B. S. Atal, "Efficient vector quantization of LPC parameters at 24 bits/frame," *IEEE Trans. Speech Audio Processing*, vol. 1, pp. 3–14, Jan. 1993.
[5] M. Barlaud *et al.*, "Guest editorial: Special issue on vector quantization," *IEEE Trans. Image Processing*, vol. 5, pp. 197–201, Feb. 1996.
[6] N. M. Nasrabadi and R. A. King, "Image coding using vector quantization: A review," *IEEE Trans. Commun.*, vol. 36, pp. 957–971, Aug. 1988.
[7] Y. Linde, A. Buzo, and R. M. Gray, "An algorithm for vector quantization design," *IEEE Trans. Commun.*, vol. COM-28, pp. 84–95, Jan. 1980.
[8] S. P. Lloyd, "Least-squares quantization in PCM," *IEEE Trans. Inform. Theory*, vol. IT-28, pp. 129–137, Mar. 1982.
[9] D. Lee, S. Baek, and K. Sung, "Modified k-means algorithm for vector quantizer design," *IEEE Signal Processing Lett.*, vol. 4, pp. 2–4, Jan. 1997.
[10] M. R. Anderberg, *Cluster Analysis for Applications*. New York: Academic, 1973.
[11] H. Robbins and S. Monro, "A stochastic approximation method," *Ann. Math. Statist.*, vol. 22, pp. 400–407, 1951.
[12] A. Benveniste, M. Metivier, and P. Priouret, *Adaptive Algorithms and Stochastic Approximations*. Berlin, Germany: Springer-Verlag, 1987.
[13] T. Kohonen, *Self-Organization and Associative Memory*. Berlin, Germany: Springer-Verlag, 1984.
[14] K. Zeger, J. Vaisy, and A. Gersho, "Globally optimal vector quantizer design by stochastic relaxation," *IEEE Trans. Signal Processing*, vol. 40, pp. 310–322, Feb. 1992.
[15] I. Katsavounidis, C. C. J. Kuo, and Z. Zhang, "A new initialization technique for generalized Lloyd iteration," *IEEE Signal Processing Lett.*, vol. 1, pp. 144–146, Oct. 1994.

A Least-Squares-Based 2-D Filtering Scheme for Stereo Image Compression

Sang-Hoon Seo, Mahmood R. Azimi-Sadjadi, and Bin Tian

***Abstract*—A two-dimensional (2-D) least squares (LS)-based filtering scheme for high fidelity stereo image compression applications is introduced in this correspondence. This method removes the effects of mismatching in a stereo image pair by applying the left image as the reference input to a 2-D transversal filter while the right image is used as the desired output. The weights of the filter are computed using a block-based LS method. A reduced order filtering scheme is also introduced to find the optimum number of filter coefficients. The principal coefficients and the disparity vectors are used together with left image to reconstruct the right image at the receiver. The proposed schemes were examined on a real stereo image pair for 3DTV applications and the results were benchmarked against those of the block-matching method.**

***Index Terms*—Least squares, stereo image compression, 2-D adaptive filtering.**

I. Introduction

Three-dimensional (3-D) video imaging has found applications in numerous areas such as 3DTV, computer games, augmented reality and surgical environments due to its capability in providing stereoscopic pictures with high resolution and great sensation of reality [1]. The compression schemes for stereo image sequences generally utilize the characteristics that there are strong spatial correlations between the right and left images as well as between the current and previous frames. The former correlation is exploited to reproduce pictures by using either the right or left image and a small amount of information that corresponds to the binocular parallax. This operation, which is similar to motion compensation generally used to predict motion in a sequence of images, is known as disparity estimation [2]–[9]. The disparity (mostly limited to the horizontal direction) vector, which requires considerably shorter length codes, will then be encoded. The decoded disparity vector is then used in conjunction with one image to generate the other one at the receiver.

Traditionally, motion estimation algorithms have been applied for disparity estimation. However, the estimation of disparity vectors requires greater accuracy compared with the estimation of motion vectors, since human eyes can see still objects sharper than moving ones and have higher resolution with 3-D images than with two-dimensional (2-D) images. In addition, there are mismatching problems between the left and right images that are caused by reflectivity/illumination differences, object occlusion, deformation,and noise that need to be compensated for.

Block-matching method is used for both disparity estimation [2], [3] as well as motion estimation [10], [11] due to its simplicity and low encoding overhead requirements. However, for disparity estimation this method has several shortcomings including blocking artifacts and lack of compensation ability for the mismatched areas. Several block-based methods were proposed [4]–[9] to provide better compensation ability with accurate disparity and/or motion estimation. The generalized block-matching method [4], [5] provides models of rotations and deformations of blocks between two images by employing the generalized spatial transformations such as affine, perspective and bilinear coordinate transformations. The phase-based methods [6], [7] use the characteristic that the phase difference in the phase domain can be related to the displacement vector between two blocks or pixels. Unlike the generalized block-matching method, these schemes are relatively insensitive to changes in the intensity and can represent displacement vector by subpixel accuracy using continuous phase information. However, these methods can not compensate for the intensity mismatching between two blocks. Baysian method [8], [9] attempts to model motion or disparity map using Markov random fields with the smoothness assumption among neighboring displacement vectors. To take care of the discontinuity of displacement vector at the object boundaries and occlusion, this method needs some *a priori* knowledge about these regions. This method does not provide compensation ability for intensity differences and further the computational effort to detect the depth discontinuity is very high.

In this correspondence, a new scheme using 2-D filtering is proposed which can be viewed as a modified version of the block-matching scheme utilizing a 2-D transversal filter to represent the effects of mismatching in stereo image pairs prior to disparity estimation. To minimize the number of filter coefficients for reconstructing the blocks, a reduced order filtering scheme is also proposed. Simulation results are presented which attest to the effectiveness of the proposed scheme in compensating for the mismatching effects when compared with the results of the standard block matching method.

II. Two-Dimensional Filtering Scheme for Disparity Estimation

The block diagram of the proposed system is shown in Fig. 1. In this scheme, the right image is considered as the desired image and the left image as the input image, X, to the 2-D transversal filter. The function of the filter is to represent the effects of mismatching between

Manuscript received June 19, 1998; revised May 22, 2000. The associate editor coordinating the review of this manuscript and approving it for publication was Prof. Rashid Ansari.

The authors are with the Signal/Image Processing Laboratory, Department of Electrical and Computer Engineering, Colorado State University, Fort Collins, CO 80523 USA (e-mail: azimi@engr.colostate.edu).

Publisher Item Identifier S 1057-7149(00)09397-0.

1057–7149/00$10.00 © 2000 IEEE

Authorized licensed use limited to: UNICAP - Universidade Católica de Pernambuco. Downloaded on October 19,2026 at 12:24:56 UTC from IEEE Xplore. Restrictions apply.